elements

Operation	Running Time	Extra Space
Insert	O(log(n))	O(1)
Top	O(log(n))	O(1)
Remove	O(log(n))§	O(n) §
Peek	O(1)	O(1)
Contains (naïve)	O(n)	O(1)
Contains	O(1) §	O(n) §
UpdatePriority	O(log(n))§	O(n) §
Heapify	O(n)	O(1)

(§) Using the advanced version of contains and maintaining an extra map from elements to indices

As with most things in computer science, time versus space is often a tradeoff. Nevertheless, sometimes there is a tendency to neglect the extra memory factor in informal analysis. With the volumes we operate since the dawn of the era of big data, however, our data structures need to hold and process billions of elements, or even more. Therefore, for instance, a fast algorithm that consumes quadratic space could be the best choice for small volumes of data, but become impractical for some real case scenarios where you need to scale out. And so, a slower algorithm using constant or logarithmic space could be the choice of election as our dataset grows.

The takeaway being: it’s even more important to take the memory factor into account as early as possible when we design a system that needs to scale.

For heaps, consider that we naturally need constant extra space per element, and linear extra space in total, to host a